

or another name.

Once you show me that, I'll give you the **exact 2–3 lines** to modify in `RiskDashboard.jsx` so the request becomes:

```
HTTP
Authorization: Bearer <JWT>
```

and your React dashboard will receive the real Spring Boot data. =€

YOU

before moving to react give me complete notes of today whatever i learned of springboot for interview purpose and give all answers of interview ready questions

CHATGPT

Absolutely. ~~Before~~ Before moving to React, let's close **Day 6 Spring Boot completely**.

Today you moved from basic Spring Security/JWT into the **Risk Engine + database persistence + analytics APIs + exception handling** of your ShiftShield project.

Below are your **complete Day 6 notes**, written specifically for **interviews + project explanation**.

~~DAY~~ DAY 6 — Spring Boot Complete Interview Notes

1. What We Built Today

Today we built the backend **Risk Management Engine** for your hospital staff-management project.

The overall flow is now:

```
TEXT
Client / React / Postman
“
JWT Authentication
“
Spring Security
“
Risk Controller
“
Risk Service
“
```

```
Risk Calculation Engine
```

```
"
```

```
RiskAssessment Entity
```

```
"
```

```
MySQL
```

```
"
```

```
Dashboard / Analytics APIs
```

The Risk Engine considers:

```
TEXT
```

```
Hours Worked
```

```
Consecutive Shifts
```

```
Rest Hours
```

```
Department Workload
```

```
Experience
```

and produces:

```
TEXT
```

```
Risk Score
```

```
+
```

```
Risk Level
```

where:

```
TEXT
```

```
LOW
```

```
MEDIUM
```

```
HIGH
```

2. RiskRequest DTO

We created a request DTO called:

```
JAVA
```

```
RiskRequest
```

It contains:

```
JAVA
```

```
private int hoursWorked;
```

```
private int consecutiveShifts;
```

```
private int restHours;
```

```
private int departmentWorkload;
```

```
private int experienceYears;
```

Why use a DTO?

Interview answer:

A DTO, or Data Transfer Object, is used to transfer data between the client and server without directly exposing the entity. It allows us to control what data the API accepts or returns and is also useful for validation.

For example:

JSON

```
{
  "hoursWorked": 10,
  "consecutiveShifts": 5,
  "restHours": 6,
  "departmentWorkload": 85,
  "experienceYears": 1
}
```

3. Validation Using Jakarta Validation

We added validation annotations:

JAVA

```
@Min(0)
@Max(24)
private int hoursWorked;

@Min(0)
@Max(14)
private int consecutiveShifts;

@Min(0)
@Max(24)
private int restHours;

@Min(0)
@Max(100)
private int departmentWorkload;

@Min(0)
private int experienceYears;
```

And in the controller:

JAVA

```
@Valid @RequestBody RiskRequest request
```

What does `@Valid` do?

Interview answer:

`@Valid` tells Spring Boot to validate the incoming request body according to the validation annotations defined in the DTO.

For example:

```
TEXT
hoursWorked = -10
```

violates:

```
JAVA
@Min(0)
```

so Spring throws:

```
TEXT
MethodArgumentNotValidException
```

and our global exception handler returns:

```
JSON
{
  "status": 400,
  "message": "Invalid risk input",
  "errors": {
    "hoursWorked": "must be greater than or equal to 0"
  }
}
```

4. Why Validation Is Important

Without validation, a client could send:

```
JSON
{
  "hoursWorked": -100,
  "restHours": -50,
  "departmentWorkload": 500
}
```

which doesn't make business sense.

Validation provides:

```
TEXT
Input
"
Validation
"
Valid ' Business Logic
Invalid ' 400 Bad Request
```

5. Risk Calculation Algorithm

This was the main business logic we implemented today.

Hours Worked

```
TEXT
<= 8 ' 0 points
<= 10 ' 10 points
> 10 ' 20 points
```

Meaning:

```
TEXT
8 hours ' 0
10 hours ' 10
11 hours ' 20
```

Consecutive Shifts

```
TEXT
<= 2 ' 0 points
<= 4 ' 10 points
> 4 ' 20 points
```

Rest Hours

TEXT

```
>= 12 ' 0 points
>= 8 ' 10 points
< 8 ' 20 points
```

Notice this is reversed because **less rest means more risk**.

Department Workload

TEXT

```
<= 60 ' 0 points
<= 80 ' 10 points
> 80 ' 20 points
```

Experience

TEXT

```
>= 5 ' 0 points
>= 2 ' 5 points
< 2 ' 10 points
```

Less experience means higher risk.

6. Risk Level Calculation

After calculating all individual scores:

TEXT

Total Risk Score

"

0 - 29 LOW

30 - 59 MEDIUM

60+ HIGH

Example

Suppose:

TEXT

Hours Worked = 10 ' 10

Consecutive Shifts = 5 ' 20

```
Rest Hours = 6 ' 20
Workload = 85 ' 20
Experience = 1 ' 10
```

Total:

```
TEXT
10 + 20 + 20 + 20 + 10
= 80
```

Therefore:

```
TEXT
80 ' HIGH
```

7. Why Put Risk Logic in Service Layer?

Our controller calls:

```
JAVA
return riskService.calculateRisk(staffId, request);
```

instead of putting all calculations inside the controller.

Interview answer:

Controllers should mainly handle HTTP requests and responses, while business logic should be placed in the service layer. This makes the application easier to maintain, test and reuse.

Architecture:

```
TEXT
Controller
"
Service
"
Repository
"
Database
```

8. Finding Staff Before Risk Calculation

We changed the Risk Service to verify that the staff member exists:

JAVA

```
staffRepository.findById(staffId)
.orElseThrow(() -> new RuntimeException("Staff not found"));
```

This prevents creating a risk assessment for a nonexistent staff member.

For example:

TEXT

```
POST /api/risk/calculate/1
```

when staff ID **1** doesn't exist gives:

TEXT

```
Staff not found
```

9. RiskAssessment Entity

This was one of the biggest additions today.

We created:

JAVA

```
@Entity
public class RiskAssessment
```

It stores:

TEXT

```
id
staffId
staffName
hoursWorked
consecutiveShifts
restHours
departmentWorkload
experienceYears
riskScore
riskLevel
```

Why create a separate entity?

Because we don't just want to calculate the risk temporarily.

We want to maintain **risk history**.

For example:

TEXT

Staff 5

Assessment 1 ' HIGH

Assessment 2 ' MEDIUM

Assessment 3 ' HIGH

This allows managers to analyze how risk changes over time.

10. Why Store `staffId` and `staffName`?

We stored:

JAVA

```
private Integer staffId;
private String staffName;
```

inside `RiskAssessment`.

For our current MVP, this makes the historical record easy to read.

For example:

JSON

```
{
  "staffId": 5,
  "staffName": "Aniket",
  "riskScore": 80,
  "riskLevel": "HIGH"
}
```

Interview note

A more advanced production design could use a JPA relationship such as:

JAVA

```
@ManyToOne
private Staff staff;
```

But our current implementation keeps the risk assessment independent and simple.

If an interviewer asks, you can say:

In the current implementation I store the staff ID and name directly for simplicity of the risk-history model.

In a more normalized production design, I could use a `@ManyToOne` relationship with the Staff entity.

That's a good answer.

11. RiskAssessmentRepository

We created:

```
JAVA
public interface RiskAssessmentRepository
extends JpaRepository<RiskAssessment, Integer>
```

This gives us built-in CRUD operations.

We also created custom methods:

```
JAVA
long countByRiskLevel(String riskLevel);

List<RiskAssessment> findByRiskLevel(String riskLevel);

List<RiskAssessment> findByStaffId(Integer staffId);
```

12. Spring Data JPA Derived Query Methods

This is an important interview topic.

We didn't manually write SQL for:

```
JAVA
findByRiskLevel()
```

Spring Data JPA understands the method name and generates the query.

For example:

```
JAVA
findByRiskLevel("HIGH")
```

conceptually means:

```
SQL
SELECT *
FROM risk_assessment
WHERE risk_level = 'HIGH';
```

Similarly:

```
JAVA
findByStaffId(5)
```

means:

```
SQL
SELECT *
FROM risk_assessment
WHERE staff_id = 5;
```

13. countByRiskLevel

We created:

```
JAVA
long countByRiskLevel(String riskLevel);
```

This is used for the dashboard.

For example:

```
JAVA
countByRiskLevel("HIGH");
```

returns:

```
TEXT
number of HIGH assessments
```

This allows the dashboard to dynamically calculate:

```
TEXT
High Risk
Medium Risk
Low Risk
```

instead of hardcoding values.

14. Saving Risk Assessment

After calculation:

```
JAVA
RiskAssessment assessment = new RiskAssessment(...);

riskAssessmentRepository.save(assessment);
```

This persists the result into MySQL.

The flow is:

```
TEXT
RiskRequest
"
Calculate score
"
Create RiskAssessment
"
repository.save()
"
Hibernate/JPA
"
MySQL
```

15. Hibernate Created the Table

After adding the entity and using:

```
PROPERTIES
spring.jpa.hibernate.ddl-auto=update
```

Hibernate created:

```
TEXT
risk_assessment
```

in our database.

Our database now contains:

```
TEXT
springboot_training

users
staff
risk_assessment
```

Interview question

What is `ddl-auto=update`?

Answer:

`spring.jpa.hibernate.ddl-auto=update` tells Hibernate to automatically update the database schema based on the entity definitions without deleting existing data.

Common values include:

```
TEXT
none
validate
update
create
create-drop
```

For production, schema migrations such as Flyway or Liquibase are generally preferred.

16. Risk Dashboard API

We created:

```
HTTP
GET /api/risk/dashboard
```

It returns:

```
JSON
{
  "totalStaff": 5,
  "totalAssessments": 3,
  "highRisk": 1,
  "mediumRisk": 1,
  "lowRisk": 1
}
```

17. RiskDashboardResponse DTO

We created:

```
JAVA
public class RiskDashboardResponse
```

with:

JAVA

```
private long totalStaff;
private long totalAssessments;
private long highRisk;
private long mediumRisk;
private long lowRisk;
```

The service calculates these values dynamically.

JAVA

```
long totalStaff = staffRepository.count();
long totalAssessments = riskAssessmentRepository.count();

long highRisk =
riskAssessmentRepository.countByRiskLevel("HIGH");

long mediumRisk =
riskAssessmentRepository.countByRiskLevel("MEDIUM");

long lowRisk =
riskAssessmentRepository.countByRiskLevel("LOW");
```

18. Why Dashboard Uses a DTO?

Instead of returning database entities directly, we return:

TEXT

RiskDashboardResponse

Interview answer:

DTOs allow us to expose only the data required by the client rather than exposing our persistence entities directly. This improves API design and reduces coupling between the database model and API response.

19. High-Risk Staff API

We created:

HTTP

GET /api/risk/high-risk

It fetches:

JAVA

```
findByRiskLevel("HIGH")
```

and converts the results into:

JAVA

```
HighRiskStaffResponse
```

containing:

TEXT

```
staffId
staffName
riskScore
riskLevel
```

Example:

JSON

```
[
{
"staffId": 5,
"staffName": "Aniket",
"riskScore": 80,
"riskLevel": "HIGH"
}
]
```

20. Why Convert Entity to DTO?

We used:

JAVA

```
assessments.stream()
.map(assessment -> new HighRiskStaffResponse(...))
.collect(Collectors.toList());
```

Interview answer:

We convert entities into DTOs so that the API exposes only the required fields and does not directly expose the database entity structure.

This is especially useful when the entity later contains:

TEXT

```
password  
internal fields  
relationships  
audit information
```

that shouldn't be exposed to clients.

21. Java Stream Used Today

We used:

```
JAVA  
stream()
```

and:

```
JAVA  
map( )
```

Interview answer

`map( )` transforms each object into another object.

For example:

```
TEXT  
RiskAssessment  
" map( )  
HighRiskStaffResponse
```

Then:

```
JAVA  
collect(Collectors.toList())
```

converts the stream back into a List.

22. Department-Wise Risk Analysis

We created:

HTTP

GET /api/risk/department-analysis

The purpose is to determine risk by department.

For example:

TEXT

Emergency

Total Assessments = 5

HIGH = 2

MEDIUM = 2

LOW = 1

Neurology

Total Assessments = 3

HIGH = 1

MEDIUM = 1

LOW = 1

Our service:

1. Gets all risk assessments.
2. Finds the corresponding staff.
3. Gets the department.
4. Groups assessments by department.
5. Counts HIGH/MEDIUM/LOW.
6. Creates `DepartmentRiskResponse`.

23. Why Department Analysis Is Useful

This is important for your **project explanation**.

Imagine a hospital manager sees:

TEXT

Emergency Department

HIGH RISK = 8

ICU

HIGH RISK = 2

General Ward

HIGH RISK = 1

The manager can identify that the Emergency department may have:

TEXT

```
excessive workload
long shifts
insufficient rest
```

and take action.

So your application isn't just calculating individual risk.

It provides **management-level decision support**.

24. Risk History API

We created:

```
HTTP
GET /api/risk/history/{staffId}
```

Example:

```
HTTP
GET /api/risk/history/5
```

The service first verifies that staff exists:

```
JAVA
staffRepository.findById(staffId)
    .orElseThrow(() -> new RuntimeException("Staff not found"));
```

Then:

```
JAVA
return riskAssessmentRepository.findByStaffId(staffId);
```

This gives all stored assessments for that staff member.

25. Why Risk History Is Important

Suppose Staff 5 has:

```
TEXT
Monday ' LOW
Tuesday ' MEDIUM
Wednesday ' HIGH
Thursday ' HIGH
```

The manager can identify that the employee's risk is increasing.

This can support:

```
TEXT
shift adjustment
rest allocation
workload reduction
manager intervention
```

26. Global Exception Handling

We created:

```
JAVA
@RestControllerAdvice
public class GlobalExceptionHandler
```

This handles exceptions globally.

Instead of writing:

```
JAVA
try {
...
} catch (...) {
...
}
```

inside every controller, we can centralize exception handling.

27. @RestControllerAdvice

Interview answer:

@RestControllerAdvice provides centralized exception handling across REST controllers. It allows us to define common exception-handling logic in one class and return consistent API responses.

28. @ExceptionHandler

We created:

JAVA

```
@ExceptionHandler(MethodArgumentNotValidException.class)
```

This handles validation failures.

And:

JAVA

```
@ExceptionHandler(RuntimeException.class)
```

handles our runtime exceptions such as:

TEXT

```
Staff not found
```

29. Validation Exception Flow

When the client sends:

JSON

```
{
  "hoursWorked": -10
}
```

the flow is:

TEXT

```
POST /api/risk/calculate/5
"
@RequestBody
"
@Valid
"
@Min(0) fails
"
MethodArgumentNotValidException
"
GlobalExceptionHandler
"
HTTP 400
```

Response:

JSON

```
{
  "status": 400,
  "message": "Invalid risk input",
```

```
"errors": {  
  "hoursWorked":  
    "must be greater than or equal to 0"  
}
```

30. HTTP Status Codes We Used

200 OK

Successful GET/request.

400 Bad Request

Client sent invalid input.

Example:

```
TEXT  
hoursWorked = -10
```

401 Unauthorized

Authentication is missing or invalid.

Think:

```
TEXT  
Who are you?
```

403 Forbidden

User is authenticated but doesn't have permission.

Think:

```
TEXT  
I know who you are,  
but you aren't allowed to do this.
```

404 Not Found

Requested resource doesn't exist.

Example:

TEXT

```
staffId = 1
```

when staff 1 doesn't exist.

31. JWT + Risk API Flow

This connects Day 5 and Day 6.

When React/Postman calls:

HTTP

```
GET /api/risk/dashboard
```

with:

HTTP

```
Authorization: Bearer <JWT>
```

the flow is:

TEXT

```
Request
```

```
"
```

```
JwtAuthenticationFilter
```

```
"
```

```
Extract JWT
```

```
"
```

```
Extract username
```

```
"
```

```
Load UserDetails
```

```
"
```

```
Validate JWT
```

```
"
```

```
Create Authentication
```

```
"
```

```
SecurityContext
```

```
"
```

```
Spring Security authorization
```

```
"
```

```
RiskController
```

```
"
```

```
RiskService
```

```
"
```

```
Database
```

32. Why `/api/risk/**` Is Protected

Our security configuration contains:

```
JAVA
.requestMatchers("/api/risk/**")
.hasAnyRole("STAFF", "MANAGER")
```

Therefore:

```
TEXT
STAFF 'allowed
MANAGER 'allowed
Unauthenticated 'rejected
```

This ensures risk information isn't publicly accessible.

33. Complete Risk API List

Your current backend risk APIs are:

Method	Endpoint	Purpose
POST	<code>/api/risk/calculate/{staffId}</code>	Calculate and save risk
GET	<code>/api/risk/dashboard</code>	Overall risk statistics
GET	<code>/api/risk/high-risk</code>	Get high-risk staff
GET	<code>/api/risk/department-analysis</code>	Department-wise analysis
GET	<code>/api/risk/history/{staffId}</code>	Staff risk history

This is now a **proper backend feature**, not just a simple CRUD API.

34. Complete Project Architecture

Your backend currently follows approximately:

```
TEXT
Client
React / Postman
|
| HTTP + JWT
“

Security
JWT Filter
```

```

    '
    "

    Controller

    '
    "

    Service
    Risk Engine

    '
    "

    Repository

    '
    "

    MySQL

```

4 INTERVIEW-READY QUESTIONS & ANSWERS

Now the most important section.

Q1. Explain your Risk Engine project.

Answer:

I developed a risk-management module for a hospital staff-management application. The system calculates staff risk based on hours worked, consecutive shifts, rest hours, department workload and experience. Each factor contributes points to a total risk score, and based on that score the staff member is classified as LOW, MEDIUM or HIGH risk. I implemented the business logic in the service layer, persisted risk assessments using Spring Data JPA and MySQL, and exposed APIs for dashboards, high-risk staff, department-wise analysis and risk history. The APIs are protected using JWT-based Spring Security.

Q2. How does your risk calculation work?

Answer:

I assign points to each risk factor using predefined business rules. For example, working more than 10 hours gives 20 points, more than four consecutive shifts gives 20 points, less than eight hours of rest gives 20 points, workload above 80 gives 20 points, and less than two years of experience gives 10 points. I add all these points to calculate the total risk score. Scores below 30 are LOW, scores from 30 to 59 are MEDIUM and scores of 60 or above are HIGH.

Q3. Why did you put the calculation logic in the Service?

Answer:

Because risk calculation is business logic. I keep the controller responsible for handling HTTP requests and delegate the business logic to the service layer. This separation improves maintainability, testing and code organization.

Q4. Why did you create `RiskRequest`?

Answer:

`RiskRequest` is a DTO that represents the data required to calculate risk. It separates the API request structure from the database entity and allows me to apply validation rules to incoming data.

Q5. Why use `@Valid`?

Answer:

`@Valid` triggers Jakarta Bean Validation on the request DTO. It checks constraints such as `@Min` and `@Max` before the request reaches the business logic.

Q6. What happens when validation fails?

Answer:

Spring throws `MethodArgumentNotValidException`. My `GlobalExceptionHandler` catches this exception using `@ExceptionHandler` and returns a structured HTTP 400 response containing the validation errors.

Q7. Why use `@RestControllerAdvice`?

Answer:

It provides centralized exception handling for REST controllers. Instead of duplicating try-catch logic in every controller, I handle common exceptions in one global exception handler.

Q8. Why create `RiskAssessment` instead of only returning the calculated result?

Answer:

I need historical data. If I only return the calculation, the result is lost after the API response. By storing every assessment, I can provide risk history, high-risk analysis, dashboard statistics and department-wise analytics.

Q9. How are risk assessments stored?

Answer:

I created a `RiskAssessment` JPA entity and a `RiskAssessmentRepository` extending `JpaRepository`. After calculating the score, I create a `RiskAssessment` object and call `riskAssessmentRepository.save()` to persist it in MySQL.

Q10. How does Spring Data JPA generate queries?

Answer:

Spring Data JPA supports derived query methods. For example, `findByRiskLevel(String riskLevel)` is interpreted from the method name and Spring Data generates the corresponding query automatically.

Q11. What is `JpaRepository`?

Answer:

`JpaRepository` is a Spring Data JPA interface that provides common database operations such as `save`, `findById`, `findAll`, `delete` and `count`. It also supports pagination and sorting and allows us to define derived query methods.

Q12. Explain `findByStaffId()`.

Answer:

JAVA

```
List<RiskAssessment> findByStaffId(Integer staffId);
```

Answer:

This is a derived query method. Spring Data JPA identifies the `staffId` property and generates a query to retrieve all risk assessments belonging to that staff member.

Q13. How does your dashboard get its data?

Answer:

The dashboard endpoint calls the service layer. The service gets the total staff count from `StaffRepository`, total assessment count from `RiskAssessmentRepository`, and counts `HIGH`, `MEDIUM` and `LOW` assessments using `countByRiskLevel()`. These values are returned through a `RiskDashboardResponse` DTO.

Q14. Why don't you hardcode dashboard numbers?

Answer:

Because dashboard data should represent the current database state. I calculate the values dynamically using repository queries, so whenever a new assessment is created, the dashboard statistics automatically change.

Q15. Explain your high-risk API.

Answer:

The high-risk endpoint retrieves assessments using `findByRiskLevel("HIGH")`. I then convert each `RiskAssessment` entity into a `HighRiskStaffResponse` DTO using Java Streams and return the list to the client.

Q16. Why use DTOs for API responses?

Answer:

DTOs prevent direct exposure of database entities and allow me to control exactly which fields are returned by the API. They also reduce coupling between the persistence layer and API layer.

Q17. Explain your department analysis API.

Answer:

The department analysis API retrieves all risk assessments, finds the corresponding staff member for each assessment, gets the staff department, groups assessments by department, and counts total, HIGH, MEDIUM and LOW assessments for each department. The result is returned using `DepartmentRiskResponse`.

Q18. Why is department-wise analysis useful?

Answer:

It helps hospital management identify departments with higher risk concentrations. For example, if the Emergency department has significantly more HIGH-risk assessments than other departments, management can investigate workload, shift schedules and rest periods.

Q19. Explain your risk history API.

Answer:

The risk history API accepts a staff ID, first verifies that the staff exists, and then retrieves all risk assessments for that staff member using `findByStaffId()`. This allows management to see how an employee's risk has changed over time.

Q20. What happens if the staff ID doesn't exist?

Answer:

The service calls `staffRepository.findById()`. If the staff member isn't found, it throws a `RuntimeException` with the message `Staff not found`. The global exception handler converts that into an HTTP 404 response.

Q21. What is the difference between 400, 401, 403 and 404?

Answer:

- 400** means the request contains invalid data.
- > **401** means authentication is missing or invalid.
- > **403** means the user is authenticated but doesn't have permission.
- > **404** means the requested resource doesn't exist.

Easy way to remember:

TEXT

```
400 ' Bad Request
401 ' Who are you?
403 ' I know you, but you're not allowed.
404 ' It doesn't exist.
```

Q22. How is your Risk API secured?

Answer:

I use JWT-based authentication with Spring Security. The client sends the JWT in the Authorization header using the Bearer scheme. My `JwtAuthenticationFilter` extracts and validates the token, loads the user details and sets the authenticated user in the `SecurityContext`. Spring Security then checks whether the user's role allows access to the requested endpoint.

Q23. Why use JWT?

Answer:

JWT allows stateless authentication. The server doesn't need to maintain a server-side session for each user. The client sends the token with each protected request, and the server validates it.

Q24. Why use `STATELESS` session management?

Answer:

```
JAVA
SessionCreationPolicy.STATELESS
```

Answer:

Because JWT authentication is stateless. We don't want Spring Security to maintain an HTTP session for authentication. Each request carries its authentication information through the JWT.

Q25. Where is the JWT processed?

Answer:

The `JwtAuthenticationFilter` runs before `UsernamePasswordAuthenticationFilter`. It extracts the Bearer token, extracts the username, validates the token and creates an `Authentication` object with the user's authorities. It then stores that authentication in the `SecurityContext`.

Q26. Why add the JWT filter before `UsernamePasswordAuthenticationFilter`?

Answer:

We want our JWT authentication to be processed before Spring's standard username/password authentication filter. This allows Spring Security to recognize the user from the JWT before authorization decisions are made.

Q27. What is `SecurityContext`?

Answer:

`SecurityContext` stores the authentication information of the currently authenticated user. Spring Security uses it to determine who the user is and what authorities or roles they have.

Q28. What is the difference between authentication and authorization?

Answer:

Authentication determines **who the user is**, while authorization determines **what that user is allowed to do**.

Example:

```
TEXT
JWT valid
"
Authentication
"
User = manager2
```

“
Authorization
“
Does manager2 have MANAGER role?

Q29. What was the React-to-Spring Boot flow we started today?

Answer:

I created a React RiskDashboard component that calls /api/risk/dashboard using fetch() inside useEffect(). The next step is to send the JWT stored by the React application in the Authorization Bearer header, so Spring Security can authenticate the request.

Most Important Project Explanation

If an interviewer says:

"Explain what you personally implemented in this project."

Give this answer:

I implemented the backend risk-management module using Spring Boot. I created a risk calculation engine that evaluates staff workload, consecutive shifts, rest hours, department workload and experience. I used DTOs with Jakarta validation to validate the input, implemented the business logic in the service layer, and used Spring Data JPA to persist each risk assessment in MySQL. I also developed APIs for overall dashboard statistics, high-risk staff, department-wise analysis and individual staff risk history. I implemented centralized exception handling using @RestControllerAdvice and secured the APIs using JWT and Spring Security with role-based authorization. I also started integrating these APIs with a React dashboard.

That is a **strong fresher-level project explanation.** =^a

Day 6 Things You Should Be Able to Explain Without Looking

Before we move to React, make sure you can explain these 12 things:

TEXT

1. What is RiskRequest?
2. Why use @Valid?
3. How does the risk score get calculated?
4. Why create RiskAssessment?
5. How is data saved using JpaRepository?
6. What are derived query methods?
7. How does the dashboard work?
8. How does high-risk API work?

9. How does department analysis work?
10. How does risk history work?
11. How does GlobalExceptionHandler work?
12. How does JWT protect these APIs?

If you can explain those confidently, **Day 6 Spring Boot is solid for interviews.**

→ Your Day 6 Backend Progress

TEXT

DAY 6

4

“ “

Risk Engine Persistence

RiskRequest RiskAssessment
Validation Repository
Scoring MySQL
Risk Level

“

REST APIs

Calculate Risk
Dashboard
High Risk
Department Analysis
Risk History

“

Exception Handling

GlobalExceptionHandler

“

Security

JWT + Role Authorization

Day 6 Spring Boot Backend = COMPLETE

The next part is **React integration**, where we'll connect your existing React application to these APIs and eventually build the actual **Risk Dashboard UI**.

One important thing: **don't change anything in your React project yet**. We'll continue exactly from where you stopped.